Documento Unificado de Artificial World

> *Auditoría integral, protocolo de verificación independiente, tabla de claims y criterio de demostración de plenitud.*

Fecha: 2026-03-09

Propósito: reunir en un solo documento la verdad técnica y editorial de `Artificial World` para producto, auditoría, documentación y validación externa.

Principio rector: ningún claim debe clasificarse como `REAL` si no puede recorrerse la cadena `claim → archivo → ejecución → artefacto → verificación externa`.

---

1. Qué es Artificial World

Artificial World es una base técnica local y auditable para crear y observar mundos simulados donde entidades actúan por decisión explícita, memoria y persistencia, en lugar de depender de LLMs o de guiones cerrados.

No es:

- un chatbot
- una plataforma total ya cerrada
- una suite enterprise completa
- una integración plena y demostrada entre todas sus capas

Sí es, hoy:

- un motor principal con núcleo verificable
- una demo web separada
- un conjunto de verticales y superficies con grados distintos de madurez
- una metodología fuerte de trazabilidad y honestidad editorial

La filosofía central es:

no preguntes a una IA; convoca un mundo que pueda demostrar su respuesta.

2. Taxonomía obligatoria

Usar exclusivamente esta clasificación:

- `REAL` : implementado y verificable con código, ejecución o artefactos
- `DEMO` : visible o utilizable como demostración, pero no equivale al sistema completo
- `PARCIAL` : existe una parte funcional, pero no la experiencia o integración completa
- `ROADMAP` : no implementado hoy; solo intención o capa futura
- `EXTERNO` : existe fuera de este repo o fuera de la build auditada

Regla:

- si un claim cae entre dos categorías, usar siempre la más conservadora

3. Matriz de evidencia por superficie

Superficie	Estado	Evidencia	Riesgo narrativo
Motor Python	REAL	`principal.py`, `nucleo/simulacion.py`, persistencia, Modo Sombra, tests	Bajo
Persistencia mundo Python	REAL	SQLite, sistemas de persistencia	Bajo
Crónica fundacional	REAL	`cronica_fundacional.py`, artefactos JSON/MD	Bajo
Baseline seed 42	REAL (acotada)	baseline reproducible y artefactos	Medio si se infla
IA local base	REAL (acotada)	`aiCore.js`, Ollama, fallback	Medio
Sistema Chess	REAL	agentes Docker, docs, ejecución	Bajo
Export público Horizons	REAL	rutas, build, `dist`, navegación	Bajo si se presenta como entrada pública
Web fullstack	DEMO	script fullstack, motor JS propio	Medio
HeroRefuge / flujo fundador	PARCIAL	backend ligero, flujo UI, mundos ligeros	Medio
CivilizationSeed web	PARCIAL	semillas y mundos ligeros	Medio
Mission Control	PARCIAL	backend, runtime, UI	Alto

| DobackSoft en repo | DEMO | mocks, rutas y sesiones limitadas | Alto |

| FireSimulator | DEMO | ruta visible, interacción, demo jugable | Medio |

| Arena (3 en Raya, Damas) | DEMO o REAL por submódulo | `/games` + minijuegos | Bajo |

| 3 en Raya | REAL | jugable en export | Bajo |

| Damas | REAL | jugable en export | Bajo |

| Ajedrez | ROADMAP | no terminado / nulo | Bajo |

| Mystic Quest | PARCIAL | capa narrativa, sin mecánicas propias completas | Medio |

| PWA del export | PARCIAL | manifest, SW, icons, build | Medio |

| Runtime 3D | ROADMAP | no implementado | Bajo |

| Integración Python/JS | ROADMAP | no existe hoy | Bajo |

| DobackSoft comercial | EXTERNO | vive fuera de este repo | Medio |

| Plasmandoluz | EXTERNO | no visible en el repo técnico actual | Bajo |

| IA-Entre-Amigos | EXTERNO | workshop Astro educativo, repo 686f6c61, no integrado en AW | Bajo |

---

4. Contradicciones narrativas detectadas

4.1 Dualidad de identidad

- Una línea define AW como base para **civilizaciones vivas con refugios, héroes y memoria**.
- Otra línea lo empuja como **infraestructura de inteligencias coordinadas para comprender proyectos reales**.

Conclusión:

- son dos categorías de producto distintas
- la primera está mejor aterrizada en UX y tesis de producto

4.2 Mission Control

- La narrativa pública sugiere observación de habitantes y civilización
- la implementación real se acerca más a una UI operativa de agentes/runs/gateways

Conclusión:

- hoy no debe venderse como capa plena de civilización viva

4.3 DobackSoft

- El repo solo sostiene una demo vertical
- algunas capas públicas tendieron a venderlo como producto comercial completo

Conclusión:

- debe quedar siempre como `DEMO` o `EXTERNO`, según el contexto

4.4 Drift factual

Se detectaron y corrigieron desalineaciones como:

- `7` vs `9` semillas
- `10` vs `11` suites
- claims demasiado fuertes en documentos públicos

4.5 Claims prohibidos

Ejemplo:

- latencia `< 1 ms`
- “100% funcionalidades”
- “producto comercial real” cuando la evidencia en repo es solo demo

Conclusión:

- la documentación pública debe seguir la taxonomía y no sobreprometer

---

5. Qué es visión y qué es producto

| Clasificación | Superficies |

|---|---|

| **REAL** | Motor Python, persistencia SQLite, Modo Sombra, crónica, runner de tests, IA local base (acotada), Sistema Chess, export como entrada pública, `3 en Raya`, `Damas`, Hub, paper web |

| **PARCIAL** | HeroRefuge, flujo fundador, CivilizationSeed web, Mission Control, Mystic Quest, simulación visible del export, PWA del export |

| **DEMO** | Web fullstack, DobackSoft en repo, FireSimulator, Arena como contenedor, verticales mostradas sin integración plena |

| **ROADMAP** | Runtime 3D, integración Python/JS, multiplayer, cloud save completo, DobackSoft pleno dentro del repo |

| **EXTERNO** | DobackSoft comercial fuera del repo, Plasmandoluz |

---

6. Qué ya tiene fuerza real

- Motor Python con 13 acciones, memoria, persistencia y artefactos
- Baseline reproducible acotada
- Taxonomía `REAL / DEMO / PARCIAL / ROADMAP`
- Flujo fundador web parcial
- Paper y documentación con prudencia metodológica
- Export público ya corregido como entrada honesta

---

7. Qué está débil o no debe inflarse

- Mission Control sigue siendo parcial
- DobackSoft en este repo sigue siendo demo
- Mystic Quest sigue siendo superficie más narrativa que sistémica
- El export no demuestra el motor Python en plenitud
- La PWA no debe venderse como cerrada sin validación real en navegador y dominio
- La baseline no debe confundirse con validación experimental madura

---

8. ¿El export corresponde a la realidad?

Respuesta corta

Sí, pero solo si se interpreta correctamente.

Corresponde a la realidad como:

- puerta de entrada pública
- mapa honesto del ecosistema
- demostración parcial de capacidades
- capa editorial y navegable

No corresponde a la realidad si se interpreta como:

- prueba de integración total
- demostración completa del motor Python
- validación plena del sistema
- evidencia de que todo AW vive en esta web

Qué sí demuestra el export

- rutas públicas reales
- Hub honesto
- Arena con `3 en Raya` y `Damas`
- `FireSimulator` como demo visible
- clasificación visible de superficies
- paper web, repositorio y base PWA

Qué no demuestra por sí solo

- motor Python ejecutándose desde la web
- reproducibilidad fuerte del sistema completo
- integración Python/JS
- Mission Control pleno
- Hero Refuge activo en el export
- DobackSoft completo
- AW en plenitud

Veredicto

El export **corresponde a la realidad si se define como entrada pública honesta al ecosistema**.

El export **no basta para demostrar AW en plenitud**.

---

9. Cómo debería explicarse públicamente

Una frase

Artificial World es un motor local de simulación multiagente con decisión por utilidad, memoria y persistencia; su capa web pública funciona como entrada honesta al ecosistema y como demo parcial de algunas superficies.

Tres líneas

- Motor Python real: agentes, memoria, persistencia, Modo Sombra.
- Web pública: Hub, Arena, paper, repositorio y demos visibles.
- Todo claim debe clasificarse como `REAL`, `DEMO`, `PARCIAL`, `ROADMAP` o `EXTERNO`.

Lo que no debe decirse

- que toda la arquitectura AW está integrada en la web pública
- que el simulador visible ya equivale al motor determinista final
- que el paper cierra la validación experimental
- que el sistema completo ya está demostrado

---

10. Protocolo de auditoría independiente

Objetivo

Demostrar qué partes de AW son reales, cuáles son demo, cuáles son parciales y cuáles son roadmap, sin depender del relato del autor.

Regla metodológica

Cada claim debe pasar por esta cadena:

`claim → archivo → ruta/componente → ejecución → artefacto → verificación externa`

Si no puede recorrerse esa cadena, el claim no debe clasificarse como `REAL`.

Niveles de auditoría

Nivel 1. Export público

Comprobar:

- rutas del router
- componentes montados
- enlaces
- badges
- build final
- PWA, si se declara

Esto prueba la verdad pública del sitio, no la plenitud de AW.

Nivel 2. Motor principal

Comprobar:

- `principal.py`
- motor de simulación
- persistencia
- Modo Sombra
- crónica fundacional
- runner de tests

Esto prueba el núcleo real del sistema.

Nivel 3. Reproducibilidad

Un tercero debe poder:

1. clonar el repo
2. instalar dependencias
3. ejecutar el motor
4. lanzar una baseline conocida
5. obtener artefactos comparables
6. contrastar salida y documentación

Nivel 4. Honestidad editorial

Comparar:

- web
- paper
- docs
- código

Y verificar:

- no vender demo como real
- no vender parcial como completo
- no vender roadmap como implementado
- no mezclar motor real con visualización parcial

---

11. Checklist operativo para auditor externo

Export

- [ ] El router coincide con las rutas visibles
- [ ] `/` es la entrada pública principal
- [ ] `/hub` funciona como mapa del ecosistema
- [ ] `3 en Raya` es jugable
- [ ] `Damas` es jugable
- [ ] `Ajedrez` no se vende como implementado

- [ ] `FireSimulator` aparece como demo
- [ ] `Mission Control` no se vende como real si no tiene flujo visible aquí
- [ ] `Hero Refuge` no se vende como implementado si no existe en esta build
- [ ] `DobackSoft` no se presenta como suite enterprise completa
- [ ] `Paper` se presenta como resumen/contexto si eso es lo que es
- [ ] `Repositorio` apunta a destino real

Motor principal

- [ ] El motor Python se puede ejecutar
- [ ] Existe persistencia verificable
- [ ] Existe baseline reproducible definida
- [ ] Existen artefactos de salida
- [ ] Existen tests ejecutables
- [ ] La documentación no contradice la ejecución real

Coherencia

- [ ] La taxonomía se aplica con consistencia
- [ ] Los claims públicos coinciden con la implementación
- [ ] No hay claims inflados o prohibidos
- [ ] No se confunde el export con la plenitud del sistema

---

12. Tabla de claims

| Claim o superficie | Categoría | Evidencia mínima | Qué sí puede decirse | Qué no debe decirse |

|---|---|---|---|---|

| Motor Python principal | REAL | `principal.py`, motor, persistencia, tests | Núcleo real y defendible de AW | Que toda la web ejecuta ese motor |

| Persistencia SQLite del mundo | REAL | DB, sistema de persistencia | Existe persistencia real | Que todas las superficies web persisten igual |

| Modo Sombra | REAL | código + docs + ejecución | Existe como capacidad real del motor | Que está expuesto plenamente en la web pública |

| Crónica fundacional | REAL | script + artefactos | Existe baseline y artefacto reproducible | Que equivale a validación científica cerrada |

| Baseline seed 42 | REAL (acotada) | script, salida, docs | Hay baseline reproducible acotada | Que prueba generalizaciones fuertes |

| Runner de tests | REAL | suite ejecutable | Existen pruebas automáticas | Que garantizan corrección total del ecosistema |

| Sistema Chess | REAL | docs + agentes Docker + ejecución | Auditoría o sistema separado verificable | Que valida por sí solo todo AW |

| IA local base | REAL (acotada) | `aiCore.js`, fallback, integración | Existe una capa local o híbrida limitada | Que AW es un copiloto pleno |

| Export público Horizons | REAL | build, rutas, `dist`, navegación | Existe como entrada pública navegable | Que demuestra AW en plenitud |

| Landing pública | REAL | `/` en export | Puerta de entrada honesta al ecosistema | Que es la app completa |

| Hub | REAL | `/hub` | Mapa honesto de superficies | Que todas las superficies listadas están completas |

| Simulación visible del export | PARCIAL | componente visible con canvas | Capa ilustrativa o demo parcial | Que es el motor determinista real |

| HeroRefuge / flujo fundador web | PARCIAL | backend ligero + flujo UI | Flujo fundador parcial | Que es la experiencia total de civilización viva |

| CivilizationSeed web | PARCIAL | seeds y mundos ligeros | Existe capa parcial de seeds | Que equivale al sistema completo del motor |

| Mission Control | PARCIAL | backend, runtime, UI | Existe una superficie parcial | Que ya es el observatorio pleno |

| Mystic Quest | PARCIAL | pantalla o capa narrativa | Existe como superficie narrativa parcial | Que es producto maduro o juego completo |

| Web fullstack general | DEMO | script de arranque y UI | Demo operativa separada del núcleo Python | Que ya unifica toda la verdad del sistema |

| FireSimulator | DEMO | ruta, UI, interacción | Demo visible y jugable | Que es simulación integrada completa |

| Arena de minijuegos | DEMO o REAL por submódulo | `/games` | Existe como superficie demostrativa | Que valida el comportamiento global de AW |

| 3 en Raya | REAL | jugable en export | Juego visible y funcional | Que demuestra el motor principal |

| Damas | REAL | jugable en export | Juego visible y funcional | Que demuestra el motor principal |

| Ajedrez | ROADMAP | no terminado / nulo | Intención o hueco futuro | Que ya está implementado |

| DobackSoft en este repo | DEMO | mocks, vertical limitada | Demo vertical conectada al ecosistema | Que es la suite enterprise completa |

| DobackSoft comercial | EXTERNO | producto fuera del repo | Existe fuera de este entorno auditado | Que está implementado aquí |

| Paper web del export | REAL | `/paper` | Resumen web y contexto técnico | Que es prueba experimental cerrada |

| Preprint técnico serio | REAL | paper + docs + límites explícitos | Posicionamiento metodológico defendible | Que es paper científico cerrado |

| Paper científico consolidado | PARCIAL o ROADMAP | faltaría evidencia más dura | Puede aspirar a ello | Que ya lo es hoy |

| Trazabilidad radical | REAL | taxonomía + docs + arquitectura | Propiedad metodológica fuerte | Que garantiza por sí sola validez experimental |

| Taxonomía REAL/DEMO/PARCIAL/ROADMAP | REAL | docs + uso consistente | Marco editorial y técnico diferenciador | Que ya se aplica perfecto sin auditoría |

| Runtime 3D | ROADMAP | no implementado | Capa futura | Que ya existe |

| Integración total Python/JS | ROADMAP | no existe hoy | Dirección futura | Que ya hay single source of truth |

| Multiplayer | ROADMAP | no implementado | Idea futura | Que se puede usar |

| Cloud save completo | ROADMAP | no implementado | Idea futura | Que existe hoy |

| API pública funcional | PARCIAL o ROADMAP | según endpoint y acceso real | Puede haber piezas internas | Que hay plataforma pública cerrada |

| PWA del export | PARCIAL | manifest, SW, icons, build | Base PWA implementada | Que ya está validada plenamente en producción |

| Plasmandoluz dentro de AW técnico | EXTERNO | no visible en repo | No forma parte del ecosistema técnico auditado actual | Que está integrado en AW |

| IA-Entre-Amigos en repo | EXTERNO | workshop Astro, 33 slides, no integrado | Material educativo externo incluido en repo | Que es superficie REAL o DEMO de AW |

---

13. Claims permitidos hoy

- “AW tiene un motor principal real y auditable.”
- “El export público funciona como puerta de entrada honesta al ecosistema.”
- “La web pública muestra demos, superficies parciales y rutas verificables.”

- “Existe una baseline reproducible acotada.”
- “La taxonomía `REAL / DEMO / PARCIAL / ROADMAP` forma parte del proyecto.”
- “El paper actual funciona mejor como preprint técnico serio que como paper científico cerrado.”

---

14. Claims que deben rebajarse o prohibirse

- “Todo AW está implementado en la web pública.”
- “La web demuestra AW en plenitud.”
- “Mission Control ya representa plenamente la narrativa de civilización viva.”
- “DobackSoft aquí es el producto enterprise completo.”
- “El simulador visible del export es el motor determinista final.”
- “El paper ya cierra la validación experimental.”
- “La baseline canónica demuestra por sí sola dinámica emergente robusta.”
- “Existe integración total Python/JS.”
- “La PWA ya está completamente validada en producción” sin prueba real de instalación.

---

15. Cómo demostrar AW en plenitud

No basta con enseñar la web.

AW solo puede decirse demostrado en plenitud si se verifican conjuntamente:

1. el motor principal real
2. la baseline reproducible
3. los artefactos verificables
4. la documentación coherente
5. el export público honesto
6. la clasificación correcta de claims

Si una de esas capas falla, la formulación correcta debe ser otra:

- núcleo real con demos parciales
- baseline fundacional reproducible
- ecosistema en construcción
- preprint técnico con trazabilidad fuerte

No:

- todo está integrado
- todo está validado
- el sistema completo está demostrado

Kit mínimo de evidencia

- `commit` exacto auditado
- script de arranque del motor real
- script de baseline canónica
- outputs esperados
- tests ejecutados
- tabla de claims con clasificación
- export construido
- correspondencia entre export y repo real

---

16. Recomendación final

La fuerza de AW no está en aparentar cierre total.

La fuerza de AW está en poder decir, con precisión:

- qué es real
- qué es demo
- qué es parcial
- qué es roadmap
- qué queda fuera de esta build

Si este documento se mantiene vivo y alineado con código, docs y export, AW gana algo más valioso que una narrativa grandiosa: **credibilidad verificable**.